# Nityanand Mathur

Github | LinkedIn | nityanandmathur@gmail.com | 7247412358 | Google Scholar

## Education

**Indian Institute Of Information Technology Guwahati**
*B. Tech in Computer Science & Engineering* — *GPA: 8.16/10*

## Publications

**SonoEdit: Null-Space Constrained Knowledge Editing for LLM-based TTS**
*Ayush Pratap Singh, Harshit Singh, Nityanand Mathur, Akshat Mandloi, Sudarshan Kamath* — *CPAL 26*

**SVGCraft: Beyond Single Object Text-to-SVG Syn. with Compr've Canvas Layout** — Project
*Ayan Banerjee, Nityanand Mathur, Josep Lladós , Umapada Pal, Anjan Dutta* — *WACV 26*

**DiffuseKronA: Param. Efficient Finetuning Method for Personalized Diff'sn Models** — Project
*Shyam Marjit, Harshit Singh, Nityanand Mathur, Sayak Paul, Chia-Mu Yu, Pin-Yu Chen* — *WACV 25*

**CLIPDrawX: Primitive-based Explanations for Text Guided Sketch Synthesis** — Project
*Nityanand Mathur, Shyam Marjit, Abhra Chaudhuri, Anjan Dutta* — *CVPR-W 25*

## Patents

**A robust approach for on-demand generation of physically plausible imagery with specific localized objects using spatial attention**
*Nityanand Mathur, Koutav Mullick, Sonam Singh, Amit Arvind Kale* — *App. 202541029889*

**A method to generate physically plausible imagery**
*Nityanand Mathur, Koustav Mullcik, Deepanker Singh, Amit Arvind Kale* — *App. 202541033217*

## Work Experience

**Smallest AI** | *Data Scientist* — October 2024 - Present
- Designed and built **Lightning V3.1**, an ultra-expressive, instruction-following real-time TTS model, achieving ∼120ms TTFB with support for 20 concurrent streams through optimized serving and inference infrastructure.
- Created **Lightning V2**, a multilingual, real-time, streaming flow-matching-based voice cloning TTS system with ∼100ms TTFB and 0.02 RTF, supporting 16 languages.
- Built an ultra-fast (<3ms) production-grade multilingual **text normalization service** from scratch, handling complex TTS edge cases and linguistic variations across multiple languages.
- Optimized **Lightning V1**, redesigning inference and streaming pipelines to reduce end-to-end latency from 280–380ms to 140–280ms, significantly improving real-time production performance.

**Bosch Research** | *Machine Learning Intern* — Jan 2024 - August 2024
- Created novel algorithms for synthetic dataset generation by object augmentation using latent diffusion models.
- Created parameter-efficient label preserving source-free domain adaptation and localised editing pipelines.

**University of Surrey** | *Research Intern - Dr. Anjan Dutta* | *Project Page* | *Paper* — Jan, 2023 – December, 2023
- Worked on introducing explainability to CLIP-based models using simple primitives, with an LDM-powered initialization for faster convergence. Introduced *Primitive-level Dropout* for noiseless sketch synthesis.

**IBM** | *Research Intern - Dr. Pin-Yu Chen* | *Project Page* | *Paper* — June, 2023 - December, 2023
- Worked on adding parameter-efficient *Kronecker Product* based adapters to personalized T2I models that are ∼35% more efficient than SOTA, while generating images with high fidelity and text-alignment.

**Osaka University** | *Data Science Research Intern — Dr. Manas Kala* | — July, 2023 - Sept, 2023
- Applied counterfactual machine learning to thermal comfort dataset to simulate the comfort of students in winter conditions to find the impact of clothing, age, grade and gender. − *Work undergoing at univ.*

## Tools

**PipScope** | *Python, Developer Tools* — 2025
- Built a lightweight TUI tool to inspect, analyze, and debug Python environments **interactively**.

**PyTorch Builder** | *Python, Computer Vision*
- Created a an interactive tool for visualizing, debugging, and profiling PyTorch models with support for layer-wise inspection, live model graph exploration, customizable visualizations, and efficient tracking of tensor operations.

**DINO Explorer** | *Python, Computer Vision*
- Created a powerful tool designed to explore and visualize DINOv2 embeddings. Given a list of folders containing images, DINO Explorer extracts their DINO embeddings and creates an interactive visualization using Voxel51.

## Skills

**Languages**: Python, SQL, Bash, C, Java, LaTeX
**Frameworks/Libraries**: PyTorch, TensorFlow, Keras, Pandas, NumPy, Scikit Learn, and OpenCV
**Tools**: Docker, Git/GitHub, Unix Shell, PyTest, Weights and Biases, DVC, Hydra.cc, Hugging Face, AWS, Gradio